

OSSP (Operating Systems And Systems Programming) - 25CS2104E

Roll No. : 2520030027

Name: Venkata Sai Krishna

SEC-8

SKILL-10:

Aim: To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

PROCEDURE :

1. PWD and Exit Built-in Program :

```
venkatsaikrishna5@VSK-HP:~$ mkdir skill10
venkatsaikrishna5@VSK-HP:~$ cd skill10
venkatsaikrishna5@VSK-HP:~/skill10$ nano pwd_exit.c
venkatsaikrishna5@VSK-HP:~/skill10$ gcc pwd_exit.c -o pwd_exit
venkatsaikrishna5@VSK-HP:~/skill10$ ./pwd_exit
shell> pwd
/home/venkatsaikrishna5/skill10
shell> hello
Invalid command: hello
shell> ls
Invalid command: ls
shell> exit
Exiting shell...
```

2. Export Built-in Program :

```
venkatsaikrishna5@VSK-HP:~/skill10$ nano export.c
venkatsaikrishna5@VSK-HP:~/skill10$ gcc export.c -o export
venkatsaikrishna5@VSK-HP:~/skill10$ ./export
shell> export NAME=Venkata Sai Krishna
Exported NAME=Venkata Sai Krishna
shell> export COLLEGE=KLH Bachupally
Exported COLLEGE=KLH Bachupally
shell> export 123ABC=hello
Invalid variable name
shell> exit
Exiting shell...
```

Code :

```
GNU nano 8.7.1          pwd_exit.c *
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <limits.h>
#include <string.h>
int main()
{
    char command[100];
    char path[PATH_MAX];
    while (1)
    {
        printf("shell> ");
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';
        if (strcmp(command, "pwd") == 0)
        {
            if (getcwd(path, sizeof(path)) != NULL)
                printf("%s\n", path);
            else
                perror("getcwd");
        }
        else if (strcmp(command, "exit") == 0)
        {
            printf("Exiting shell...\n");
            exit(0);
        }
        else if (strlen(command) == 0)
        {
            continue;
        }
        else
        {
            printf("Invalid command: %s\n", command);
        }
    }
    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

int valid_name(char *name)
{
    int i;

    if (!(isalpha(name[0]) || name[0] == '_'))
        return 0;

    for (i = 1; name[i] != '\0'; i++)
    {
        if (!(isalnum(name[i]) || name[i] == '_'))
            return 0;
    }

    return 1;
}

int main()
{
    char command[200];
    char *name;
    char *value;
    char *equal;

    while (1)
    {
        printf("shell> ");
        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
```

```

    {
        printf("Exiting shell...\n");
        break;
    }

    if (strncmp(command, "export ", 7) == 0)
    {
        char temp[200];

        strcpy(temp, command + 7);

        equal = strchr(temp, '=');

        if (equal == NULL)
        {
            printf("Usage: export NAME=VALUE\n");
            continue;
        }

        *equal = '\0';

        name = temp;
        value = equal + 1;

        if (!valid_name(name))
        {
            printf("Invalid variable name\n");
            continue;
        }

        if (setenv(name, value, 1) == 0)
            printf("Exported %s=%s\n", name, value);
        else
            perror("setenv");
    }
    else if (strlen(command) == 0)
    {
        continue;
    }
    else
    {
        printf("Invalid command: %s\n", command);
    }
}

return 0;
}

```

Observation :

1. The pwd command successfully displays the current working directory.
2. The exit command successfully terminates the program.
3. Invalid commands are detected and an error message is displayed.
4. The export NAME=VALUE command successfully creates environment variables.
5. Existing environment variables can be updated with new values.
6. Invalid variable names are rejected.
7. Incorrect export syntax is handled with an appropriate error message.

Result :

The PWD-Exit and Export built-in programs were successfully implemented and executed. The programs correctly performed directory retrieval, exit handling, environment variable management, validation, and error handling.